

Pinky 라이브러리 LLM fine tuning 하기

Pinky Contents

25 May, 2026

Table of Contents

1	파인 튜닝 전 데이터 생성	4
1.1	데이터 생성하는데 참고할 pinky 라이브러리 관련 md파일 다운로드	4
1.2	md 파일을 참고하여 다음과 같은 형태로 학습용 pinky_finetune_data.json 파일 생성하기.....	5
1.3	또는 ChatGPT에게 아래 내용으로 질문해서 데이터 생성하기	5
1.4	생성된 데이터들	6
1.5	데이터가 부족할 경우 더 요청하면 더 생성 가능하다	6
1.6	생성된 데이터들을 pinky_finetune_data.json 파일을 만들어서 추가하기	7
1.7	생성된 데이터	8
2	파인 튜닝	9
2.1	파인튜닝에 필요한 라이브러리들 설치	9
2.2	라이브러리 импорт 및 설정 (Configuration)	9
2.3	모델 로드 (Load Model)	9
2.4	LoRA 어댑터 추가	10
2.5	데이터셋 준비 및 포매팅 (Prepare Dataset)	10
2.6	Alpaca 포맷	11
2.7	학습 실행 (Train).....	12
2.8	모델 저장 (Save Model).....	12
2.9	모델 저장관련 참고	13
2.10	코드 실행 해서 학습 시작	13
2.11	코드 실행 후 아래 같은 화면이 나온다면 3 입력.....	14
2.12	학습 완료.....	15
2.13	Modelfile과 gguf 파일 생성되었는지 확인	15
2.14	gguf 파일 이름 변경	16
2.15	Modelfile 수정	16
2.16	ollama에 모델 등록	17
2.17	ollama list 로 잘 등록되어 있는지 확인	18
2.18	ollama run 으로 파인튜닝한 모델 동작해보기	19
2.19	파인 튜닝 전 모델과 비교	21
3	참고.....	22

3.1	아래와 같은 에러가 날 경우	22
3.2	참고) unsloth으로 받은 모델 위치	23
3.3	파인튜닝 가이드	23
3.4	참고 유튜브	23

1 파인 튜닝 전 데이터 생성

1.1 데이터 생성하는데 참고할 pinky 라이브러리 관련 md파일 다운로드



```

1
2 # Pinky Library Documentation (Full)
3
4 > 본 문서는 Pinky 전용 파이썬 라이브러리의 API 레퍼런스입니다. 모듈별로 대표 클래스와 메서드를 설명하며, 예제 코드는 최소 실행 단위로 정리되어 있습니다.
5
6 ## Ultrasonic
7
8 ### `get_dist()`
9
10 * **Description**
11 | Pinky의 초음파 센서로 거리를 측정한 값을 반환한다.
12 * **Syntax**
13 | `get_dist()`
14 * **Parameters**
15 | None
16 * **Returns**
17 | `int distance` : 초음파센서로부터 받은 거리 (단위: cm 가정)
18 * **Example Code**
19
20 ```python
21 from pinkylib import Ultrasonic
22 import time
23
24 pinky = Ultrasonic()
25
26 try:
27     while True:
28         dist = pinky.get_dist()
29         print(f'{dist:.2f} cm')
30         time.sleep(1)
31     finally:
32         pinky.close()
33 ```
34
35 ---
36
37 ## Buzzer
38
39 ### `buzzer_start()`
40
41 * **Description**
42 | Pinky의 부저 핀에 대해 PWM 신호를 시작한다.

```

1.2 md 파일을 참고하여 다음과 같은 형태로 학습용 pinky_finetune_data.json 파일 생성하기

```

({} pinky_finetune_data.json ×
llm > {} pinky_finetune_data.json > {} 8
1
2
3
4   "input": "배터리 전압이랑 잔량 좀 확인해줘.",
5   "output": "from pinkylib import Battery\n\nbattery = Battery()\n\ntry:\n    voltage = battery.get_voltage()\n    percentage = battery.battery_percentage()\n6   },
7   {
8   "input": "로봇을 1초 동안 앞으로 이동시켰다가 멈추는 코드 알려줘. 속도는 50으로 해.",
9   "output": "from pinkylib import Motor\n\nimport time\n\npinky = Motor()\n\ntry:\n    pinky.enable_motor()\n    # 앞으로 이동 (최우 모터 속도 50)\n    pinky.move(50, 50\n10  },
11  {
12  "input": "초음파 센서로 거리를 측정해서 10cm 이하면 멈추고, 아니면 계속 전진하는 코드를 짜줘.",
13  "output": "from pinkylib import Ultrasonic, Motor\n\nimport time\n\nsonar = Ultrasonic()\n\nmotor = Motor()\n\ntry:\n    motor.enable_motor()\n    while True:\n14  },
15  {
16  "input": "부저로 '도레미' 소리를 1초씩 내고 싶어.",
17  "output": "from pinkylib import Buzzer\n\nimport time\n\nbuzzer = Buzzer()\n\ntry:\n    buzzer.buzzer_start()\n    buzzer.set_buzzer_duty(50)\n    \n    # 도 (26\n18  },
19  {
20  "input": "LCD 화면을 녹색으로 3초간 채웠다가 꺼줘.",
21  "output": "from pinky_lcd import LCD\n\nfrom PIL import Image\n\nimport time\n\nlcd = LCD()\n\n# 녹색 이미지 생성 (320x240)\n\nimg = Image.new('RGB', (320, 240), color\n22  },
23  {
24  "input": "카메라를 켜서 화면을 주피터 노트북에 계속 보여주게 해줘. 10초 동안만.",
25  "output": "from pinkylib import Camera\n\nncam = Camera()\n\nncam.start()\n\n# 10초 동안 주피터 노트북에 프레임 출력\n\nncam.play_jupyter(10)\n\nncam.close()\n\n",
26  },
27  },
28  },
29  },
30  },
31  },
32  },
33  },
34  },
35  },
36  },
37  },
38  },
39  },
40  },
41  },
42  },
43  },
44  },
45  },
46  },
47  },
48  },
49  },
50  },
51  },
52  },
53  },
54  },
55  },
56  },
57  },
58  },
59  },
60  },
61  },
62  },
63  },
64  },
65  },
66  },
67  },
68  },
69  },
70  },
71  },
72  },
73  },
74  },
75  },
76  },
77  },
78  },
79  },
80  },
81  },
82  },
83  },
84  },
85  },
86  },
87  },
88  },
89  },
90  },
91  },
92  },
93  },
94  },
95  },
96  },
97  },
98  },
99  },
100 },

```

1.3 또는 ChatGPT에게 아래 내용으로 질문해서 데이터 생성하기

- 현재 무료 계정으로 작성 중
- canvas 모드 활성화 하면 바로 다운로드 할 수 있다

ChatGPT ▾

Free offer ×

Share ...

pinky_library.md
File

첨부한 pinky_library.md 파일은 'Pinky'라는 교육용 로봇을 제어하기 위한 파이썬 라이브러리 명세서입니다.
이 문서를 완벽하게 이해한 뒤, 로봇 제어용 LLM(거대언어모델)을 파인튜닝하기 위한 학습 데이터를 생성해주세요.

[요구사항]

1. ****포맷****: 반드시 아래와 같은 JSON 리스트 형식으로 작성해주세요.

```
[
  {
    "input": "사용자 질문",
    "output": "실행 가능한 파이썬 코드"
  }
]
```

2. ****내용 구성****: 총 100개의 다양한 예제를 만들어주세요.

- 단순 제어 (예: "앞으로 1초 이동해")
- 센서 활용 (예: "장애물 있으면 멈춰")
- 복합 로직 (예: "카메라로 마커 찾아서 ID 출력해")
- 로봇 설명 (예: "초음파 센서는 어떻게 써?")

3. ****코드 작성 규칙 (매우 중요)****:

- 반드시 첨부된 문서에 정의된 클래스와 함수만 사용하세요. (없는 함수 창조 금지)
- 하드웨어 안전을 위해 try-finally 블록을 사용하여 마지막에 꼭 close()를 호출하도록 작성하세요.
- 주석과 설명은 한 줄로 작성하세요.

첨부한 `pinky\_library.md` 파일은 'Pinky'라는 교육용 로봇을 제어하기 위한 파이썬 라이브러리 명세서입니다. 이 문서를 완벽하게 이해한 뒤, 로봇 제어용 LLM(거대언어모델)을 파인튜닝하기 위한 학습 데이터를 생성해주세요.

[요구사항]

1. ****포맷****: 반드시 아래와 같은 JSON 리스트 형식으로 작성해주세요.

```
[
  {"input": "사용자 질문", "output": "실행 가능한 파이썬 코드"}
]
```

2. ****내용 구성****: 총 20개의 다양한 예제를 만들어주세요.

- 단순 제어 (예: "앞으로 1초 이동해")
- 센서 활용 (예: "장애물 있으면 멈춰")
- 복합 로직 (예: "카메라로 마커 찾아서 ID 출력해")
- 로봇 설명 (예: "초음파 센서는 어떻게 써?")

3. ****코드 작성 규칙 (매우 중요)****:

- 반드시 첨부된 문서에 정의된 클래스와 함수만 사용하세요. (없는 함수 창조 금지)
- 하드웨어 안전을 위해 **try-finally** 블록을 사용하여 마지막에 꼭 `close()`를 호출하도록 작성하세요.
- 주석과 설명은 한국어로 작성하세요.

4. ****출력****: 서론이나 사족 없이 ****오직 JSON 데이터만**** 출력해주세요. 작

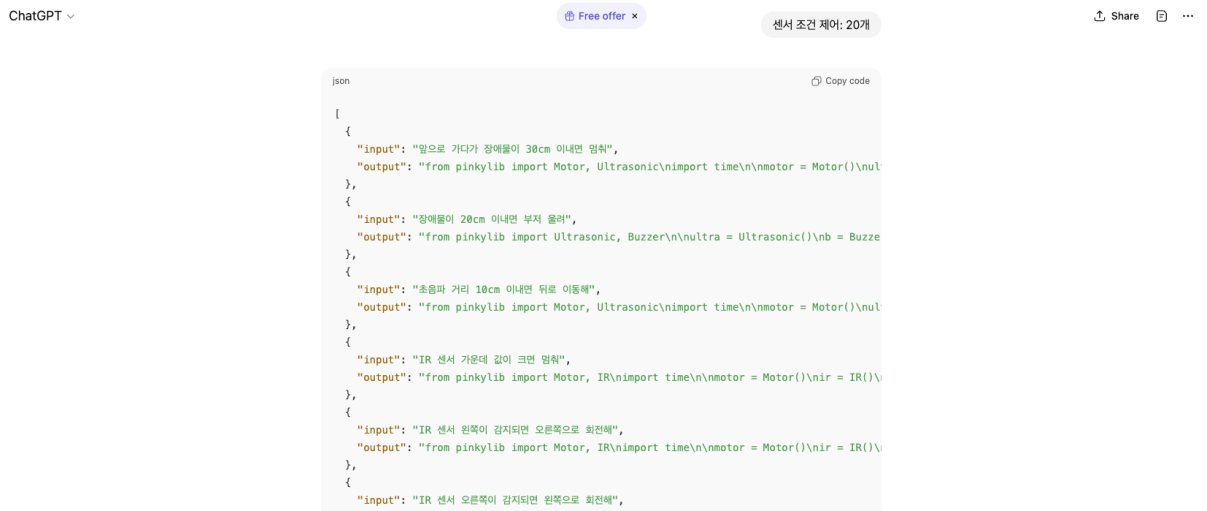
1.4 생성된 데이터들

```
json
[
  {
    "input": "앞으로 1초 이동해",
    "output": "from pinkylib import Motor\nimport time\nmotor = Motor()\ntry:\n    moto"
  },
  {
    "input": "뒤로 0.5초 이동해",
    "output": "from pinkylib import Motor\nimport time\nmotor = Motor()\ntry:\n    moto"
  },
  {
    "input": "제자리에서 왼쪽으로 회전해",
    "output": "from pinkylib import Motor\nimport time\nmotor = Motor()\ntry:\n    moto"
  },
  {
    "input": "초음파 센서 거리 출력해",
    "output": "from pinkylib import Ultrasonic\nultra = Ultrasonic()\ntry:\n    dist = "
  },
  {
    "input": "장애물이 20cm 이내면 멈춰",
    "output": "from pinkylib import Motor, Ultrasonic\nimport time\nmotor = Motor()\nul"
  },
  {
    "input": "부저 한 번 울려",
    "output": "from pinkylib import Buzzer\nb = Buzzer()\ntry:\n    b.buzzer_start()\n"
```

1.5 데이터가 부족할 경우 더 요청하면 더 생성 가능하다

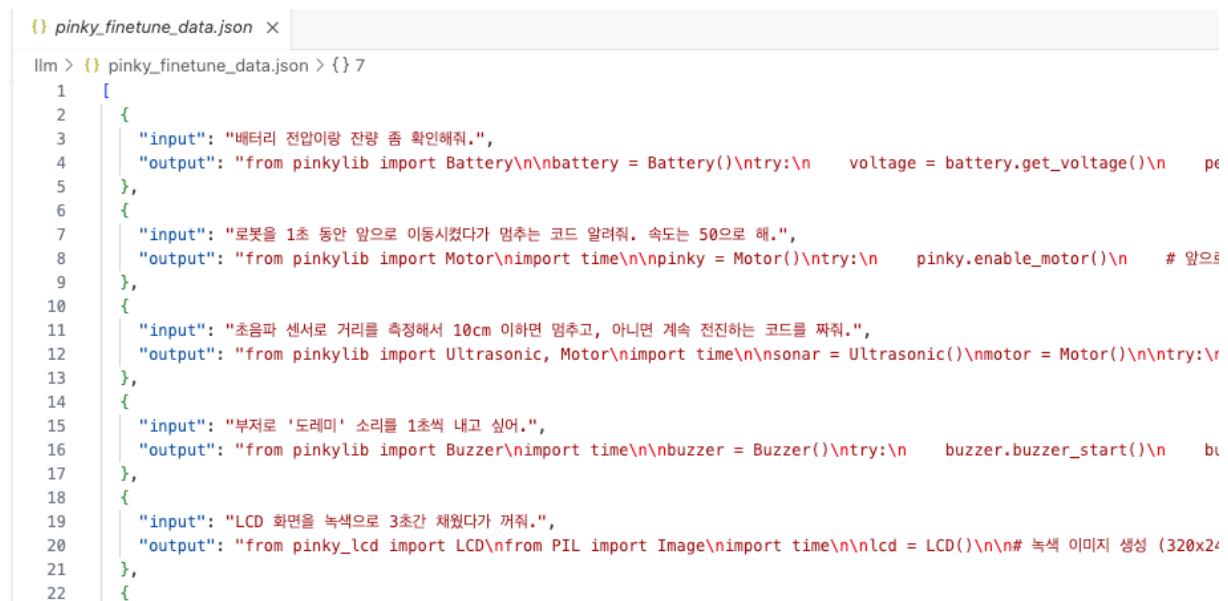
- gpt 특성상 한번에 큰 데이터 생성은 불가능 하므로 계속 요청해서 추가를 하면 많은 데이터를 얻을 수 있다

- 예)
 - 모터 제어관련 20개 추가
 - 센서 조건 제어: 20개 추가
 - 센서 + 모터 복합 로직: 20개 추가
 - 카메라 / ArUco: 20개 추가
 - 설명형 질문(코드 포함): 20개 추가

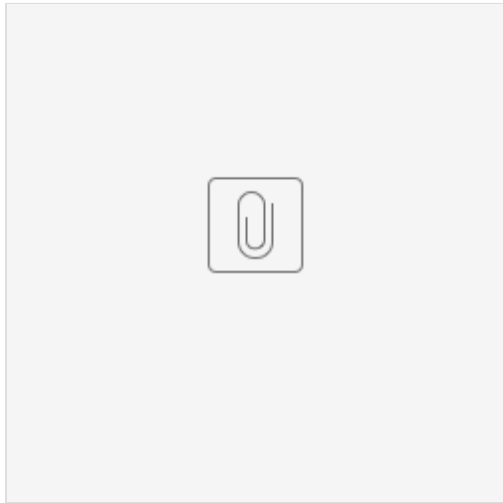


1.6 생성된 데이터들을 pinky_finetune_data.json 파일을 만들어서 추가하기

- 중요) 추가하기전에 잘못된 데이터가 없는지 확인 후 추가하기



1.7 생성된 데이터



2 파인 튜닝

2.1 파인튜닝에 필요한 라이브러리들 설치

```
pip3 install unsloth trl peft accelerate bitsandbytes
```

- `unsloth` : 학습 속도를 높이고 메모리를 절약하는 최적화 라이브러리
- `peft` : LoRA와 같은 효율적인 파인튜닝 기법을 사용하기 위한 라이브러리
- `trl` : Transformer Reinforcement Learning의 약자로, `SFTTrainer` (지도 미세 조정 트레이너)를 제공합니다.
- `bitsandbytes` : 모델을 4bit로 양자화(압축)하여 메모리 사용량을 줄여줍니다.
- `accelerate` : PyTorch 모델 학습을 가속화하고 GPU 분산 학습 등을 돕습니다.

2.2 라이브러리 임포트 및 설정 (Configuration)

```
llm > train_pinky.py
1  from unsloth import FastLanguageModel
2  import torch
3  from datasets import Dataset
4  import json
5  from trl import SFTTrainer
6  from transformers import TrainingArguments
7
8  # 1. Configuration
9  max_seq_length = 2048
10 dtype = None # Auto detection
11 load_in_4bit = True
12 input_json = "pinky_finetune_data.json"
13 model_name = "unsloth/Qwen3-4B-unsloth-bnb-4bit" # Using the one from the notebook
```

2.3 모델 로드 (Load Model)

- `max_seq_length` : 토큰 길이
- `load_in_4bit` : 4비트 양자화

```

15 # 2. Load Model
16 print("Loading Model...")
17 model, tokenizer = FastLanguageModel.from_pretrained(
18     model_name = model_name,
19     max_seq_length = max_seq_length,
20     dtype = dtype,
21     load_in_4bit = load_in_4bit,
22 )

```

2.4 LoRA 어댑터 추가

- **LoRA (Low-Rank Adaptation)**: 수억 개의 파라미터를 가진 모델 전체를 수정하는 대신, 아주 작은 추가 레이어(어댑터)만 학습시킨다
- `r = 16` : LoRA 랭크 (8, 16, 32, 64, 128 중 선택)
 - 값이 클수록 모델이 어려운 작업(harder tasks)에서 정확도를 높일 수 있지만 메모리를 더 많이 사용하고 학습 속도가 느려진다
 - 주의) 숫자가 너무크면 과적합이 발생할 수 있다

```

24 # 3. Add LoRA Adapters
25 model = FastLanguageModel.get_peft_model(
26     model,
27     r = 16, # Adjust rank if needed (16, 32, 64)
28     target_modules = ["q_proj", "k_proj", "v_proj", "o_proj",
29                       "gate_proj", "up_proj", "down_proj",],
30     lora_alpha = 16,
31     lora_dropout = 0,
32     bias = "none",
33     use_gradient_checkpointing = "unsloth",
34     random_state = 3407,
35     use_rslora = False,
36     loftq_config = None,
37 )
38

```

2.5 데이터셋 준비 및 포매팅 (Prepare Dataset)

- **시스템 프롬프트**: 모델에게 "너는 Pinky 로봇 제어 전문가야"라는 페르소나와 제약사항(특정 라이브러리 사용, 모르면 모른다고 하기)을 주입
- **포매팅**: `### Instruction: (지시) -> ### Response: (답변)` 구조의 **Alpaca 포맷**을 사용합니다.
- `tokenizer.eos_token` : 모델에게 문장의 끝을 명확히 알려주어 생성이 멈추지 않는 현상을 방지

```

39 # 4. Prepare Dataset
40 print("Preparing Dataset...")
41 with open(input_json, "r", encoding="utf-8") as f:
42     data = json.load(f)
43
44 # 시스템 프롬프트 정의
45 system_prompt = """당신은 교육용 로봇 'Pinky'를 제어하는 Python 전문가입니다.
46 사용자의 요청에 대해 `pinkylib`과 `pinky_lcd` 라이브러리를 사용하여 정확한 Python 코드를 작성하세요.
47 모르는 내용이거나 라이브러리에 없는 기능이라면 억지로 코드를 만들지 말고 솔직하게 "모르겠습니다"라고 대답하세요.
48 답변은 한국어로 친절하게 설명하세요."""
49
50 def format_prompt(example):
51     input_text = example['input']
52     output_text = example['output']
53
54     # 시스템 프롬프트
55     system_message = system_prompt
56
57     # [표준 Alpaca 포맷 적용]
58     text = f"""### Instruction:
59 {system_message}
60
61 {input_text}
62
63 ### Response:
64 {output_text}"""
65
66     return text + tokenizer.eos_token
67
68 # -----
69
70 # Convert to HuggingFace Dataset
71 formatted_data = [{"text": format_prompt(item)} for item in data]
72 dataset = Dataset.from_list(formatted_data)
73

```

2.6 Alpaca 포맷

- 모델에게 "지시(Instruction)"와 "답변(Response)"을 명확히 구분하여 가르치기 위한 표준 텍스트 형식
- 구조

```

### Instruction:
(사용자의 질문이나 명령)

### Response:
(모델이 해야 할 답변)

```

- 이 형식을 사용하면 모델이 언제 사용자의 말을 듣고, 언제 답변을 시작해야 하는지 정확하게 학습할 수 있다

2.7 학습 실행 (Train)

- **목적:** 준비된 모델과 데이터로 실제 파인튜닝을 수행합니다.
- **주요 설정:**
 - `num_train_epochs` : 전체 학습데이터 반복횟수
 - `per_device_train_batch_size` : 한 번에 학습 데이터 설정한 수만큼 GPU에 올립니다 (배치사이즈).
 - `gradient_accumulation_steps` : 설정한 수만큼 데이터를 모은 뒤 한 번에 업데이트
 - (예: 1(배치) × 4(누적) = 실제 배치 사이즈 4).

```

74 # 5. Train
75 print("Starting Training...")
76 trainer = SFTTrainer(
77     model = model,
78     tokenizer = tokenizer,
79     train_dataset = dataset,
80     dataset_text_field = "text",
81     max_seq_length = max_seq_length,
82     dataset_num_proc = 2,
83     args = TrainingArguments(
84         per_device_train_batch_size = 1,
85         gradient_accumulation_steps = 4,
86         warmup_steps = 10,
87         num_train_epochs = 10,
88         learning_rate = 2e-4,
89         fp16 = not torch.cuda.is_bf16_supported(),
90         bf16 = torch.cuda.is_bf16_supported(),
91         logging_steps = 1,
92         optim = "adamw_8bit",
93         weight_decay = 0.01,
94         lr_scheduler_type = "linear",
95         seed = 3407,
96         output_dir = "pinky_outputs",
97     ),
98 )
99
100 trainer_stats = trainer.train()
101

```

2.8 모델 저장 (Save Model)

- Ollama에서 사용하기 위해 `save_pretrained_gguf` 로 GGUF 파일로 저장

```

102 # 6. Save Model
103 print("Saving Model...")
104 # model.save_pretrained("pinky_lora_model")
105 # tokenizer.save_pretrained("pinky_lora_model")
106
107 # Merge to GGUF
108 model.save_pretrained_gguf("pinky_model", tokenizer, quantization_method = "q4_k_m")
109 print("Done!")

```

2.9 모델 저장관련 참고

- `model.save_pretrained("pinky_lora_model")` :
 - LoRA 어댑터 파일만 따로 저장하는 명령어
 - Ollama 같은 외부 툴이 아니라, Unsloth로 Python 코드에서 다시 이 모델을 불러와서(Load) 추론하거나 추가 학습을 하려고 할 때 사용합니다. 용량이 매우 작습니다.
- `tokenizer.save_pretrained("pinky_lora_model")` :
 - 토크나이저(텍스트를 숫자로 변환하는 도구)의 설정과 어휘 사전(vocab)을 저장합니다.
 - 모델을 나중에 불러올 때, 학습할 때 사용했던 토크나이저와 **똑같은 규칙**으로 텍스트를 처리해야 하기 때문입니다. 특히 새로운 특수 토큰을 추가했거나 설정을 변경했다면 반드시 함께 저장해야 합니다.
- `model.push_to_hub_gguf` :
 - 변환된 GGUF 모델을 **Hugging Face Hub**에 바로 업로드
 - 토큰 필요

2.10 코드 실행 해서 학습 시작

- 50개 데이터 학습 약 5분 소요 (4050 laptop 6GB VRAM GPU 기준)
- 모델 생성 약 10분 소요
- 총 약 15분

```
python3 train_pinky.py
```

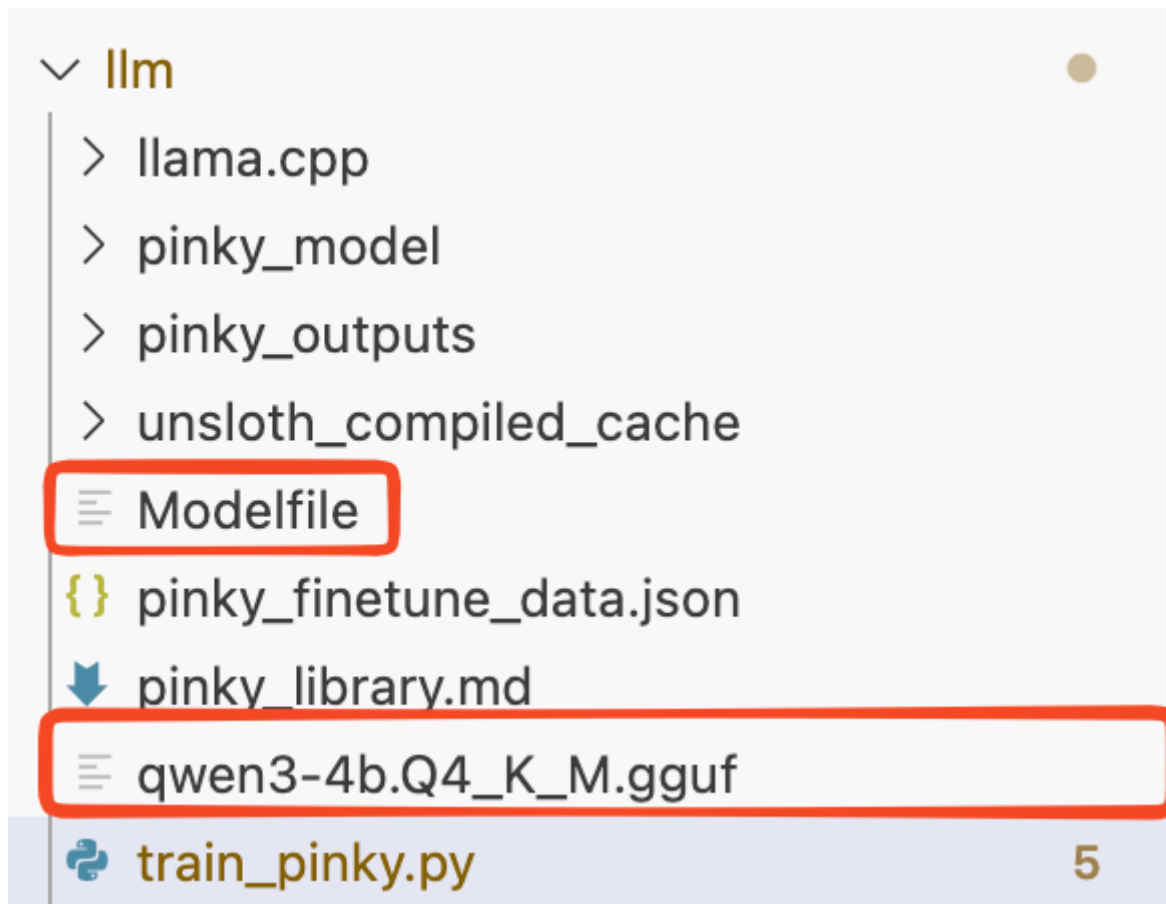

2.12 학습 완료

```

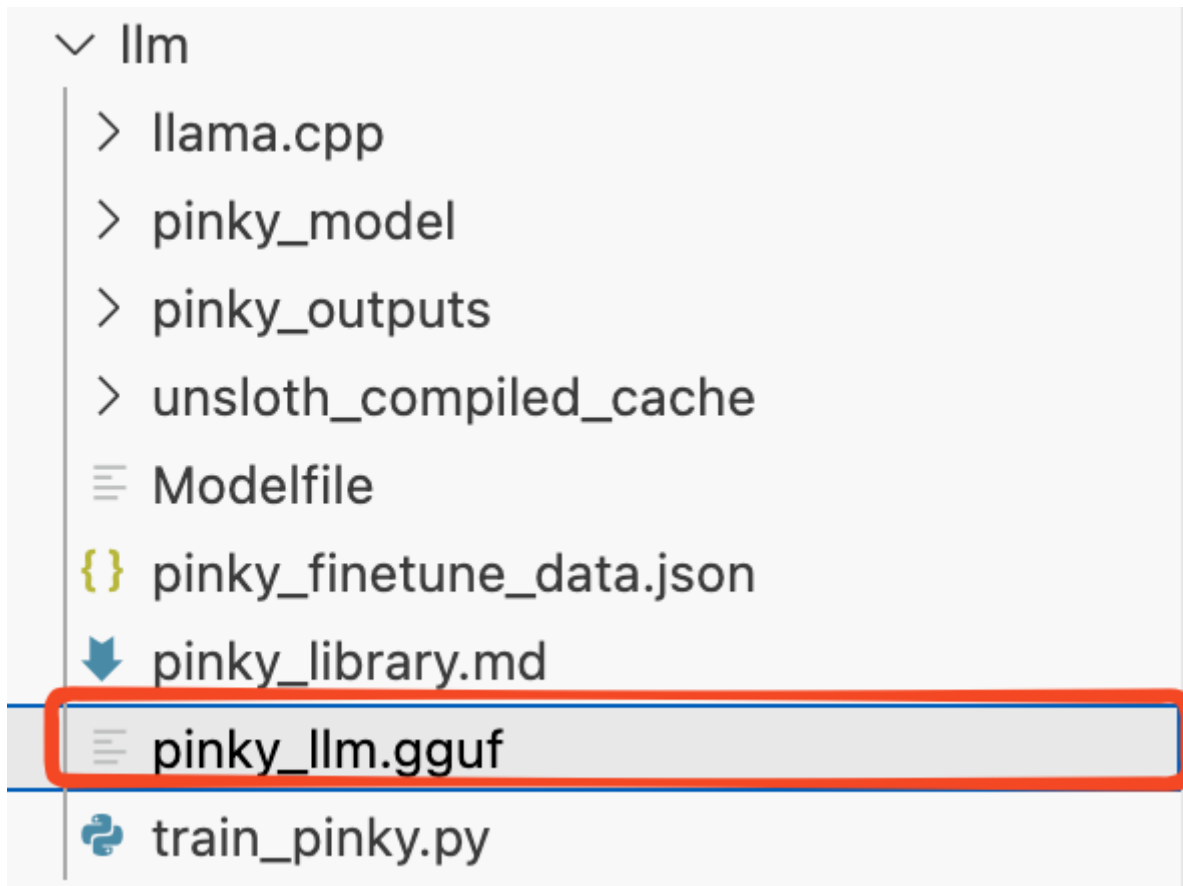
Unsloth: All required system packages already installed!
Unsloth: Install llama.cpp and building - please wait 1 to 3 minutes
Unsloth: Cloning llama.cpp repository
Unsloth: Install GGUF and other packages
Unsloth: Successfully installed llama.cpp!
Unsloth: Preparing converter script...
Unsloth: [1] Converting model into bf16 GGUF format.
This might take 3 minutes...
Unsloth: Initial conversion completed! Files: ['qwen3-4b.BF16.gguf']
Unsloth: [2] Converting GGUF bf16 into q4_K_M. This might take 10 minutes...
Unsloth: Model files cleanup...
Unsloth: All GGUF conversions completed successfully!
Generated files: ['qwen3-4b.Q4_K_M.gguf']
Unsloth: example usage for text only LLMs: llama-cli --model qwen3-4b.Q4_K_M.gguf -p "why is the sky blue?"
Unsloth: Saved Ollama Modelfile to current directory
Unsloth: convert model to ollama format by running - ollama create model_name -f ./Modelfile - inside current directory.
Done!
(llm) p13@desktop:~/llm$

```

2.13 Modelfile과 gguf 파일 생성되었는지 확인



2.14 gguf 파일 이름 변경



2.15 Modelfile 수정

```
# 1. 모델 경로 설정
FROM ./pinky_llm.gguf

# 2. 템플릿 설정 (Alpaca Style)
TEMPLATE """### Instruction:
{{ .System }}

{{ .Prompt }}

### Response:
"""

# 3. 시스템 프롬프트
SYSTEM """당신은 교육용 로봇 'Pinky'를 제어하는 Python 전문가입니다.
사용자의 요청에 대해 `pinkylib`과 `pinky_lcd` 라이브러리를 사용하여 정확한 Python 코드를 작성하세요.
하드웨어 제어 시에는 반드시 `try-finally` 구문을 사용하여 `close()`로 자원을 해제해야 합니다.
```

모르는 내용이거나 라이브러리에 없는 기능이라면 억지로 코드를 만들지 말고 솔직하게 "모르겠습니다"라고 대답하세요.
답변은 한국어로 친절하게 설명하세요. """

```
# 4. 파라미터 튜닝
# 코드 생성 모델이므로 창의성(temperature)을 낮춰 환각을 줄입니다.
PARAMETER temperature 0.2
PARAMETER top_p 0.9
PARAMETER repeat_penalty 1.1
```

```
llm > Modelfile
1 # 1. 모델 경로 설정
2 FROM ./pinky_llm.gguf
3
4 # 2. 템플릿 설정 (Alpaca Style)
5 TEMPLATE """### Instruction:
6 {{ .System }}
7
8 {{ .Prompt }}
9
10 ### Response:
11 """
12
13 # 3. 시스템 프롬프트
14 SYSTEM """당신은 교육용 로봇 'Pinky'를 제어하는 Python 전문가입니다.
15 사용자의 요청에 대해 `pinkylib`과 `pinky_lcd` 라이브러리를 사용하여 정확한 Python 코드를 작성하세요.
16 하드웨어 제어 시에는 반드시 `try-finally` 구문을 사용하여 `close()`로 자원을 해제해야 합니다.
17 모르는 내용이거나 라이브러리에 없는 기능이라면 억지로 코드를 만들지 말고 솔직하게 "모르겠습니다"라고 대답하세요.
18 답변은 한국어로 친절하게 설명하세요. """
19
20 # 4. 파라미터 튜닝
21 # 코드 생성 모델이므로 창의성(temperature)을 낮춰 환각을 줄입니다.
22 PARAMETER temperature 0.2
23 PARAMETER top_p 0.9
24 PARAMETER repeat_penalty 1.1
25 |
```

2.16 ollama에 모델 등록

```
ollama create pinky_llm.gguf -f ./Modelfile
```

```
(llm) pl3@desktop:~/llm$ ollama create pinky_llm.gguf -f ./Modelfile
gathering model components
copying file sha256:5cad0f97bed1696ecd4e2df8325db56d42e986281204de89e6b0850e50cf5ca 100%
parsing GGUF
using existing layer sha256:5cad0f97bed1696ecd4e2df8325db56d42e986281204de89e6b0850e50cf5ca
creating new layer sha256:8721274bc81e349b7a484ba06abc4f4e3de165b1bfd21cb6d344ffb5e5c75635
creating new layer sha256:e72be27dad82afe4c825a7448fad4b28657650456b94b7e0d3293e6e0008f8b3
creating new layer sha256:5339b880343957817c7fb60584fe581d03c63db1045cd2455c34c2d61104dabb
writing manifest
success
(llm) pl3@desktop:~/llm$
```

2.17 ollama list 로 잘 등록되어 있는지 확인

```
(llm) pl3@desktop:~/llm$ ollama list
NAME                                ID                                SIZE    MODIFIED
qwen3:4b                            359d7dd4bcda                    2.5 GB  2 minutes ago
pinky_llm.gguf:latest                de546c1fdb87                    2.5 GB  7 minutes ago
(llm) pl3@desktop:~/llm$
```

2.18 ollama run 으로 파인튜닝한 모델 동작해보기

```
(llm) pl3@desktop:~/llm$ ollama run pinky_llm.gguf
>>> 안녕
안녕하세요! 어떻게 도와드릴까요? ☺

>>> pinky에 대해 설명 해줘
Pinky는 Python 라이브러리 `pinkylib`과 화면 제어 라이브러리 `pinky_lcd`를 통해 전자제어, 센서 데이터 수집, LCD 화면 출력 등을 수행하는 교육용 로봇입니다. 주로 자율 주행, 사진 촬영, 거리 측정, 자세 제어 등의 교육용 프로젝트에 사용됩니다.

>>> Send a message (/? for help)
```

```

>>> 모터 동작 코드 알려줘
from pinkylib import Motor
import time

motor = Motor()
try
    motor.enable_motor()
    # 전진 (좌우 모터 각각 30, 30)
    motor.move(30, 30)
    time.sleep(2)
    # 정지
    motor.stop()
    # 회전 (좌우 모터 각각 30, -30)
    motor.move(30, -30)
    time.sleep(2)
    # 정지
    motor.stop()
    # 후진 (-30, -30)
    motor.move(-30, -30)
    time.sleep(2)
    # 정지
    motor.stop()
    # 휠 차단 (모터 비활성화)
    motor.disable_motor()
finally
    motor.close()

>>> 나이키 로고 어민오
모르겠습니다.

>>> led 코드 알려줘
from pinkylib import LED

led = LED()
try
    # 흰색으로 전체 켜기
    led.fill((255, 255, 2 Instruction:

```

2.19 파인 튜닝 전 모델과 비교

```
(llm) pl3@desktop:~/llm$ ollama run qwen3:4b-instruct-2507-q4_K_M
```

```
>>> 안녕
```

```
안녕하세요! 😊 어떻게 도와드릴까요?
```

```
>>> 핑키에 대해 설명해줘
```

```
물론이죠! 😊
```

"핑키"는 여러 가지 의미로 사용될 수 있어요. 아래에 주요 의미들을 간단히 설명해 드릴게요:

1. **핑키 (Pink) – 색상**

- **핑키**는 영어로 "****pink****"를 한국어로 직역한 말로, ****핑크색****(빨간색과 블루 사이의 풀빛의 색)을 의미해요.

- 흔히 ****사랑****, ****여성성****, ****귀여움****, ****화사함****을 상징하는 색으로 사용돼요.

- 예: "그녀의 옷은 페인트처럼 흐린 펀키 색이에요."

> 💡 흥미로운 사실: 페인트 색 중에서 펀키는 19세기 후반부터 사용되며, 초기에는 빨간색과 파란색의 중간색으로 생각되었어요.

2. **핑키 (Pink) – 캐릭터 / 인물**

- ****핑키 블루**** (Pink Blue): 이는 "핑키"와 "블루"를 조합한 이름으로, 유명한 캐릭터가 아닐 수 있어요.

- 또는 ****핑키 리프**** (Pink Liff), ****핑키 캐릭터**** 등, 애니메이션, 만화, 게임 등에서 등장하는 캐릭터들이 있어요.

- 예: ****핑키 슈퍼스타**** (예: 페인트 페인팅 캐릭터 등)

3. **핑키 (Pink) – 브랜드 / 제품 이름**

- 많은 브랜드가 펀키 색을 사용해 ****여성 중심****, ****귀여움****, ****소소한 감성****을 전달해요.

- 예: 펀키 토이, 펀키 케이크, 펀키 브랜드 의류 등

4. **핑키 (Pink) – 문화적 의미**

- ****핑키 문화****는 1960년대 이후, 특히 1970년대부터 여성의 자율성과 표현을 강조하는 문화로 발전했어요.

- 예: 펀키한 스타일, 펀키한 음악 (예: 펀키 밴드), 펀키한 댄스 등

- 펀키한 것은 "자신의 감성과 스타일을 표현하는 것"을 의미할 수도 있어요.

3 참고

3.1 아래와 같은 에러가 날 경우

- pinky_outputs, pinky_lora_model 폴더 삭제하고 다시 학습

```
During handling of the above exception, another exception occurred
:
```

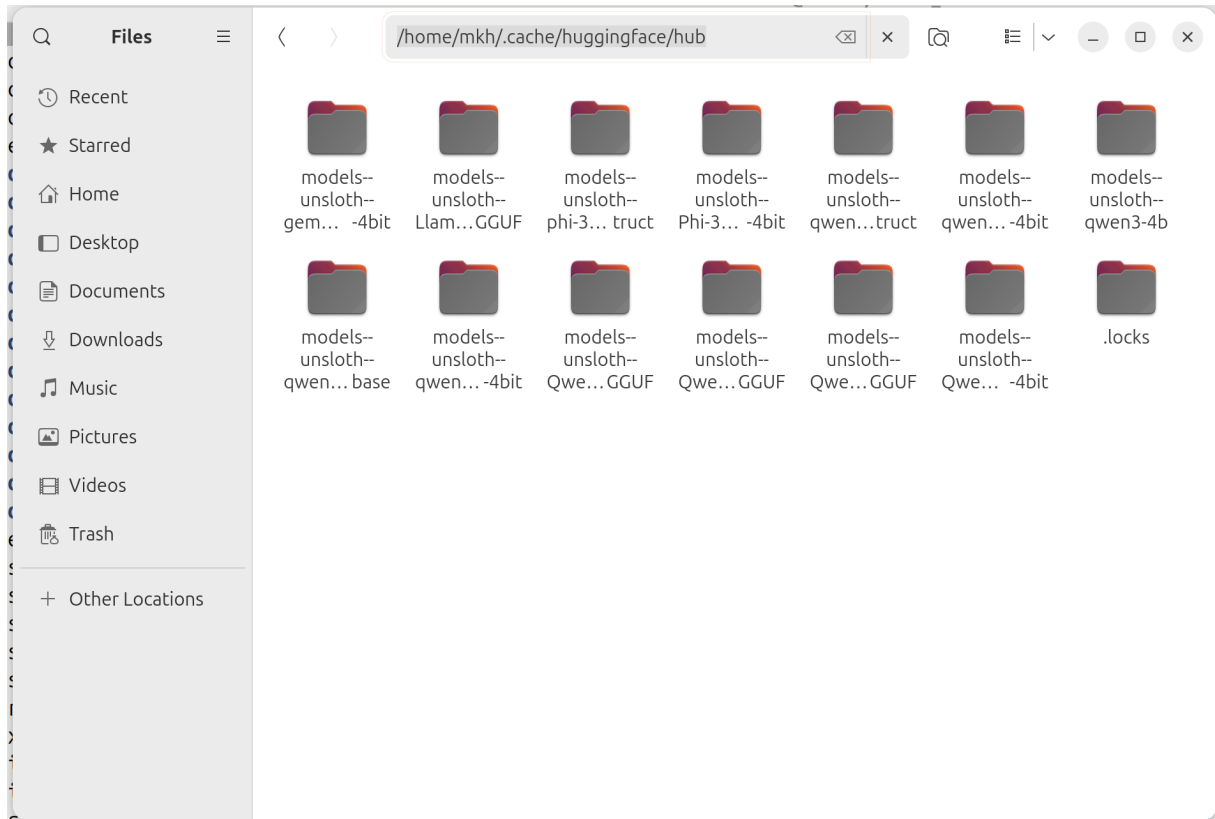
```
Traceback (most recent call last):
```

```
File "/home/mkh/ollama_ws/train_pinky.py", line 104, in <module>
    model.save_pretrained_gguf("pinky_model_gguf", tokenizer, quan
tization_method = "q4_k_m")
```

```
File "/home/mkh/venv/test/lib/python3.12/site-packages/unsloth/s
ave.py", line 1909, in unsloth_save_pretrained_gguf
```

```
    raise RuntimeError(f"Failed to save/merge model: {e}")
RuntimeError: Failed to save/merge model: Model merge failed with
error: Bad in-place call: input tensor size [3072, 8192] and outpu
t tensor size [2560, 9728] should match
```

3.2 참고) unsloth으로 받은 모델 위치



3.3 파인튜닝 가이드

- <https://unsloth.ai/docs/get-started/fine-tuning-llms-guide>
- 튜토리얼 <https://unsloth.ai/docs/get-started/fine-tuning-llms-guide/tutorial-how-to-finetune-llama-3-and-use-in-ollama>

3.4 참고 유튜브

- <https://www.youtube.com/watch?v=pTaSDVz0gok&list=TLGGPXqGnPB9E9syMDAxMjAyNg&t=11s>